NLP 243 Deep Learning for Natural Language Processing

Fall 2022

Including content adapted from Dilek-Hakkani-Tur, Vivian Chen and Jay Alammar

Final Exam

- Monday, 12/11/24, 1:30 PM
- In Class
- Short Answers, conceptual questions
  - How does improve Seq to Seq
  - Calculate TF IDF
- Dimension based Questions
- CNN/RNN/Attention/BPE/Greedy Sampling/ intermediate values based on parameters

Final Projects

- Schedule:
  - Presentation slides are due on Dec 1st 5pm.
  - A random set of teams will present on Tuesday
  - The others will present on Thursday
  - Final Project Reports and code are due December 5th, 11:55 PM (no extension)!

Outline for Today

1. Impact of Transformers on NLP (and ML more broadly)
2. Understanding the Transformer Model
3. Drawbacks and Variants of Transformers
4. BERT

Readings

- Vaswani et al, NIPS 2017. Attention is all you need.
- Continuing Ch 10 of the Dive into Deep Learning book
- Blogs:
  - Jay Alammar, The illustrated transformer
  - PyTorch explanation by Sasha Rush: http://nlp.seas.harvard.edu/2018/04/03/attention.html

Transformers: Is Attention All We Need?

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

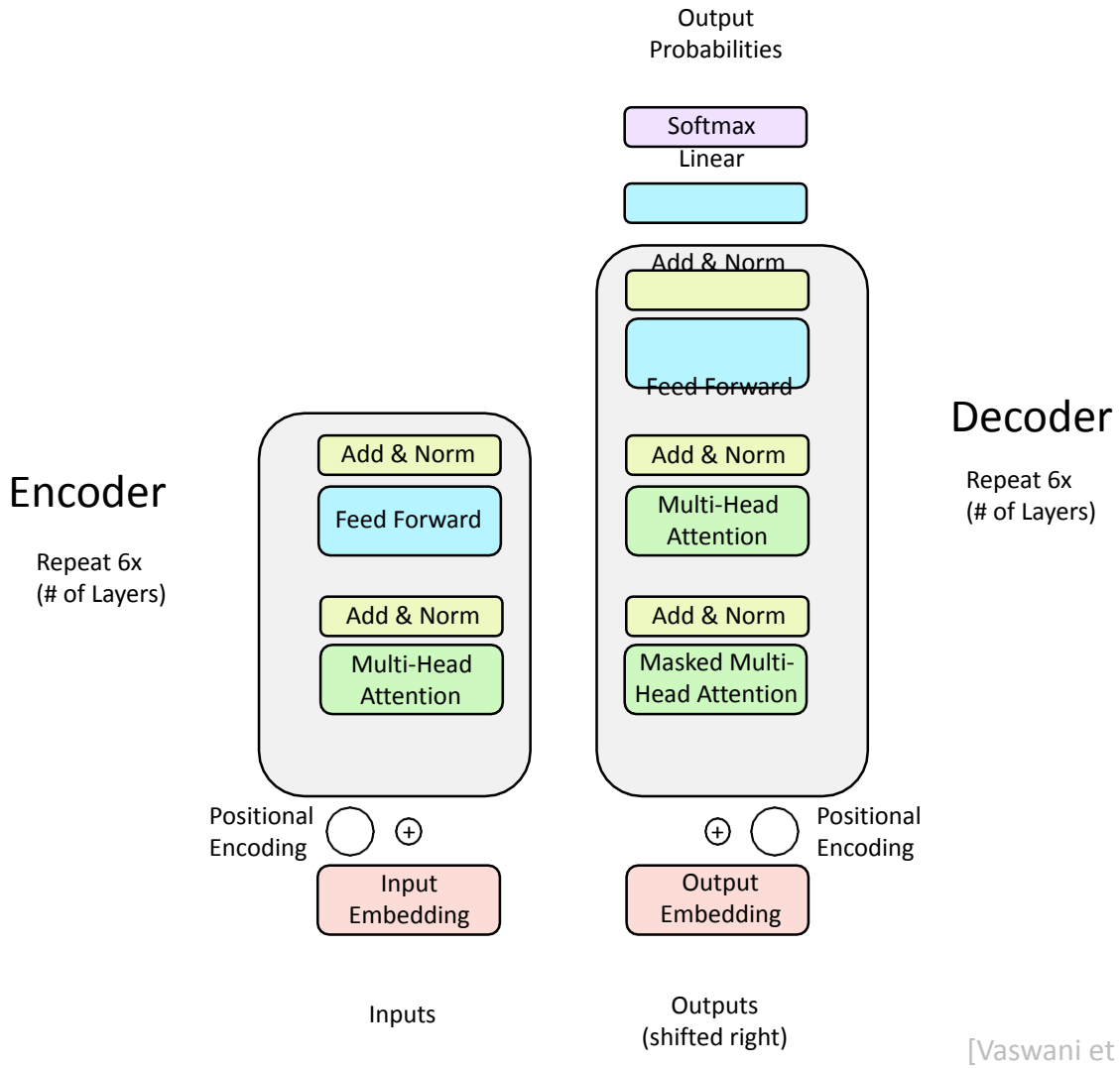
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Transformers Have Revolutionized the Field of NLP

Courtesy of Paramount Pictures



[Vaswani et al., 2017]

Great Results with Transformers: Machine Translation

First, Machine Translation results from the original Transformers paper!

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Not just better Machine Translation BLEU scores

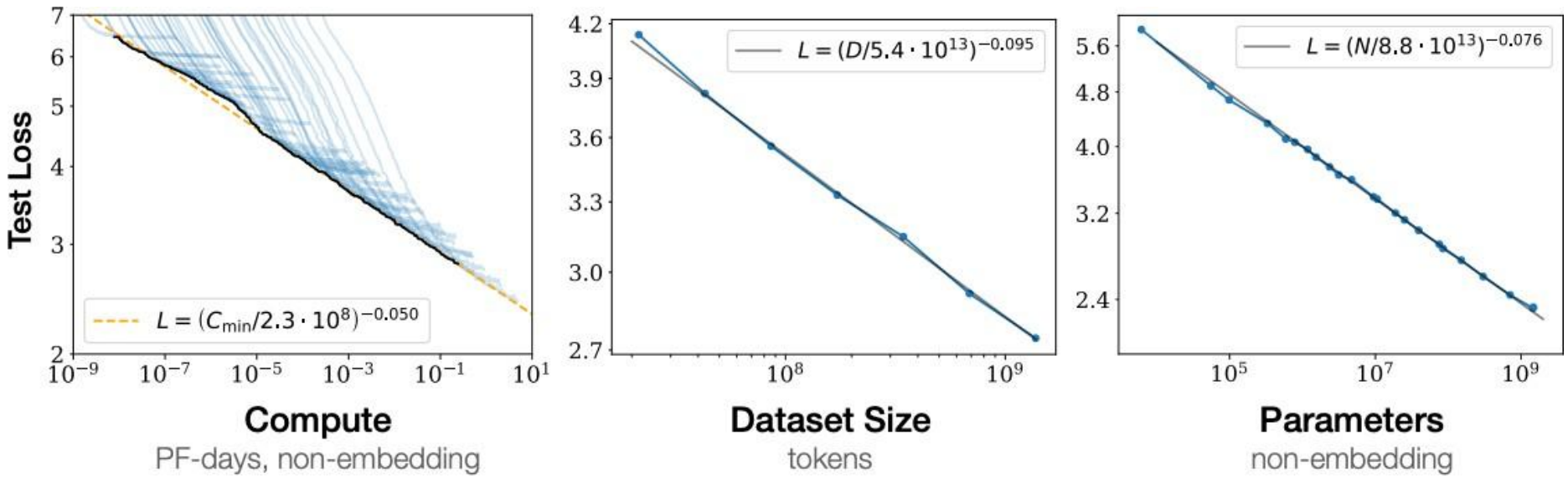
Also more efficient to train!

[Test sets: WMT 2014 English-German and English-French]

[Vaswani et al., 2017]

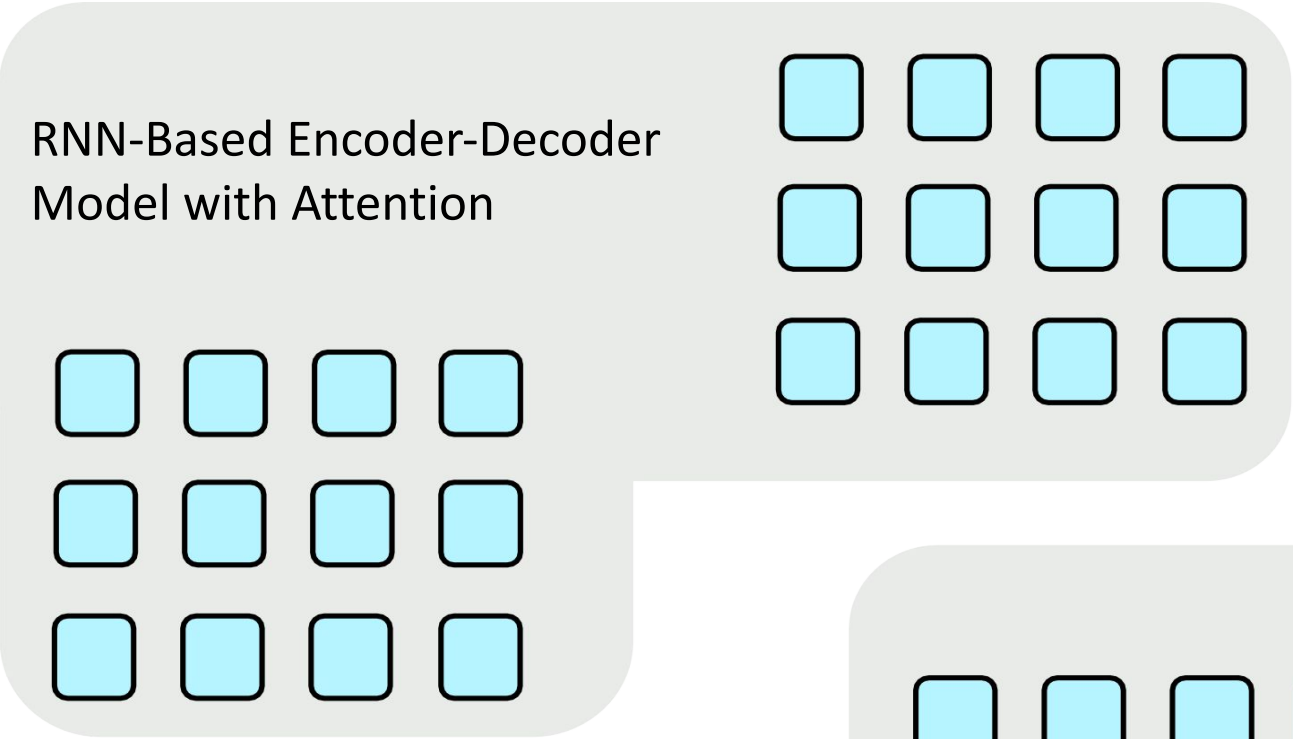
Scaling Laws: Are Transformers All We Need?

- With Transformers, language modeling performance improves smoothly as we increase model size, training data, and compute resources.
- This power-law relationship has been observed over multiple orders of magnitude with no sign of slowing!
- If we keep scaling up these models (with no change to the architecture), could they eventually match or exceed human-level performance?



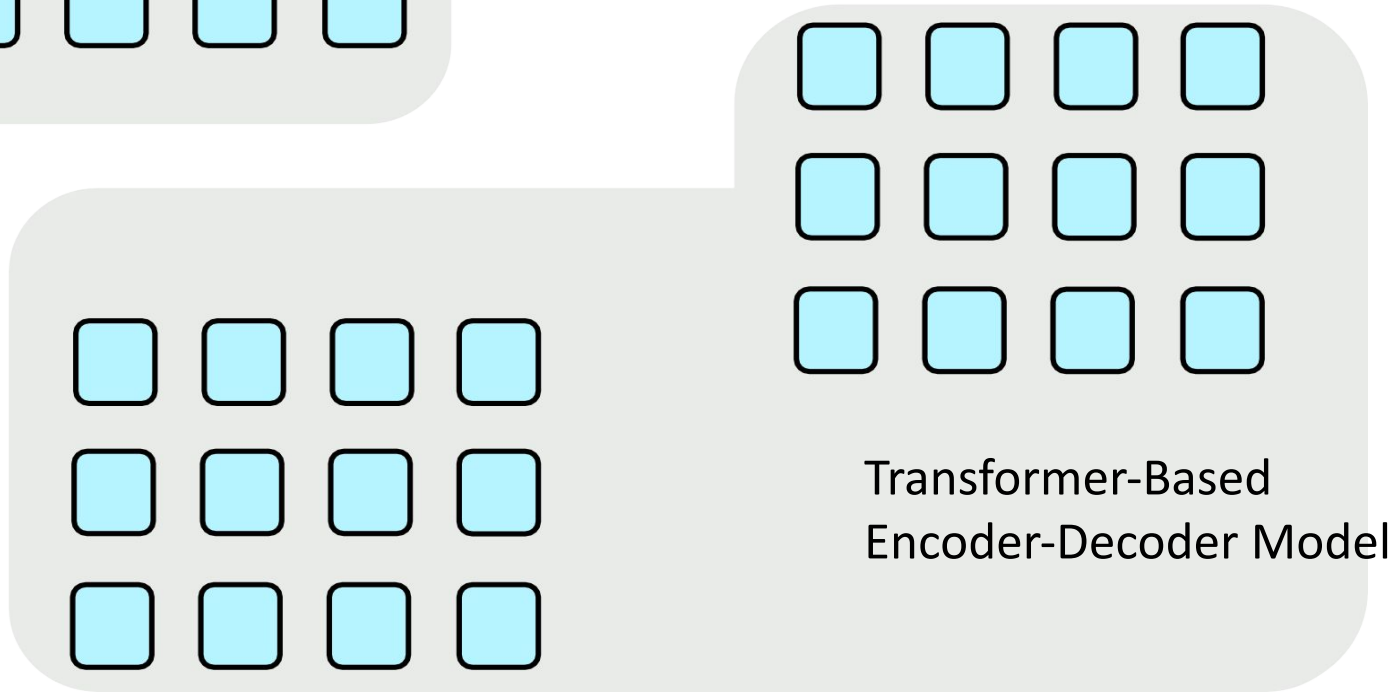
[Kaplan et al., 2020]

Computational Dependencies for Recurrence vs. Attention



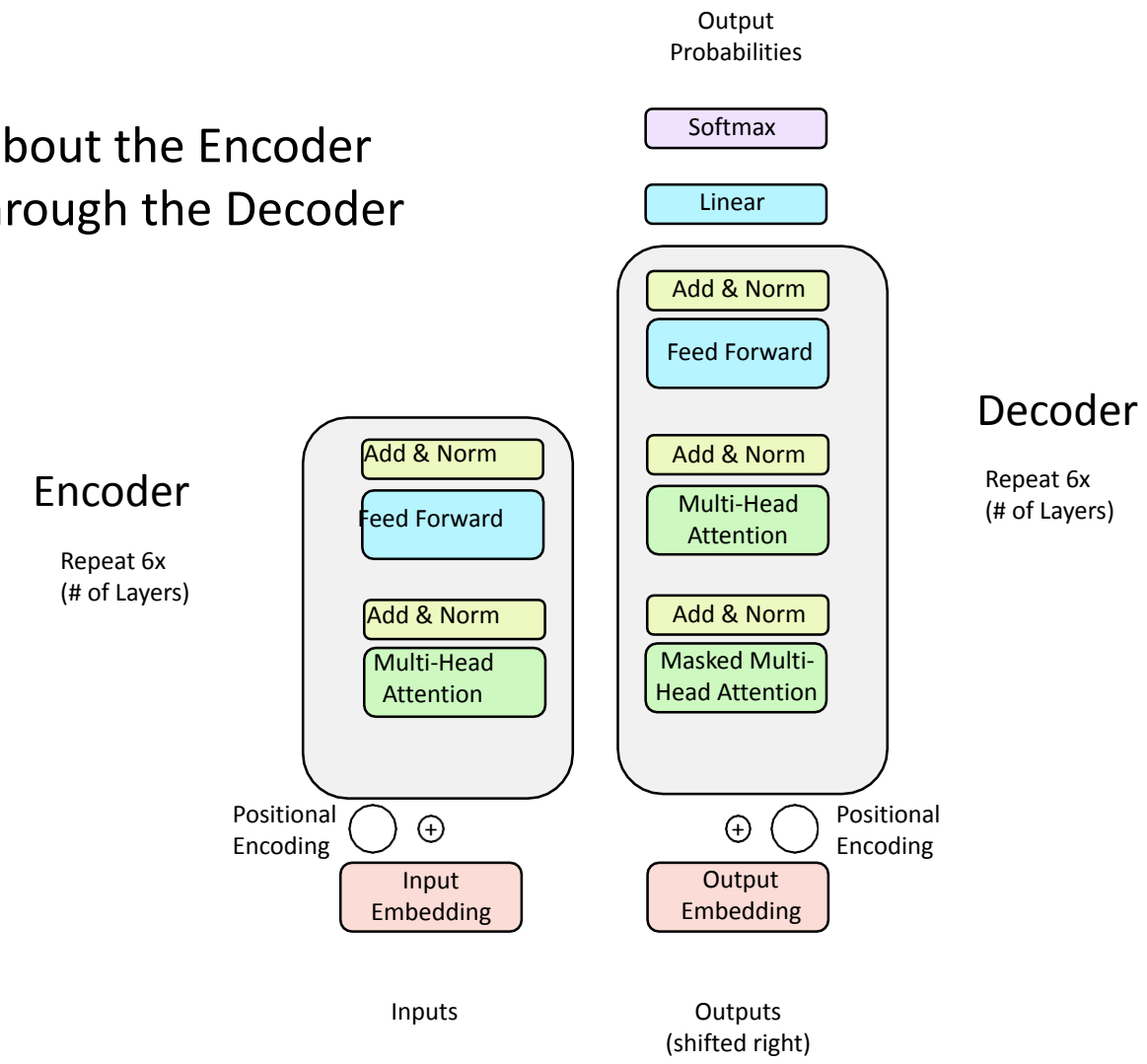
Transformer Advantages:

- Number of unparallelizable operations does not increase with sequence length.
- Each "word" interacts with each other, so maximum interaction distance: O(1).

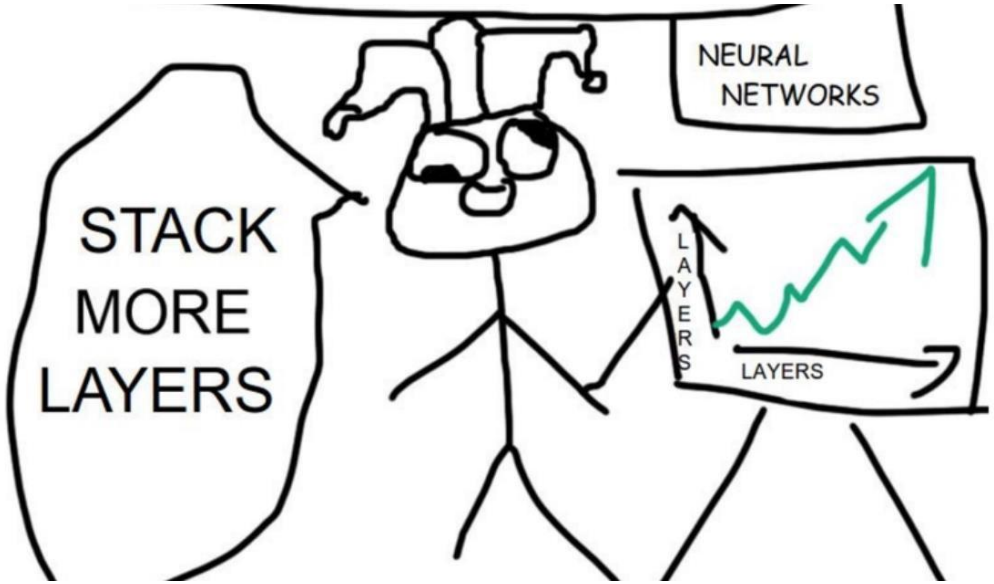


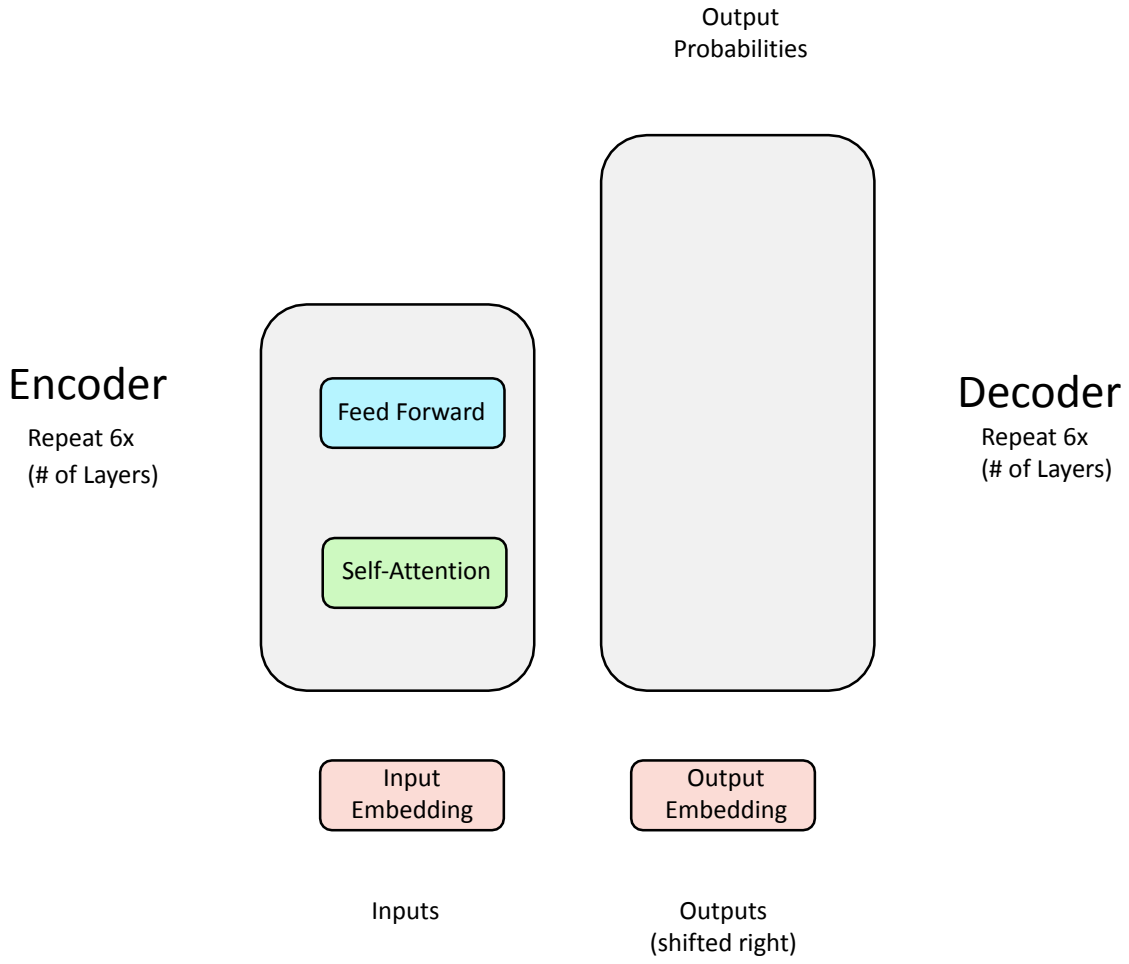
The Transformer Encoder-Decoder [Vaswani et al., 2017]

- First, we will talk about the Encoder
- Next, we will go through the Decoder



But how do we make this work for deep networks?





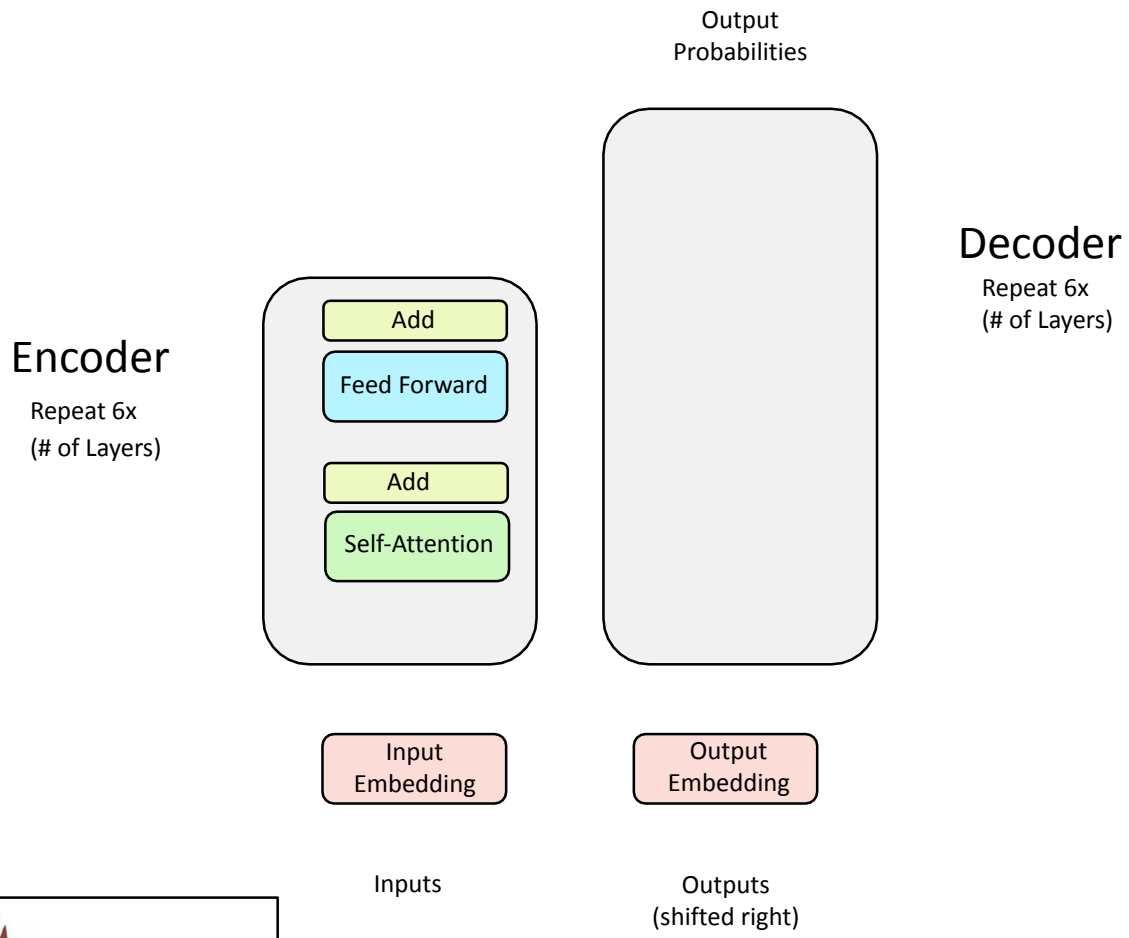
Training Trick #1: Residual Connections
Training Trick #2: BatchNorm
Training Trick #3: LayerNorm
Training Trick #4: Scaled Dot Product Attention

Training Trick #1: Residual Connections [He et al., 2016]

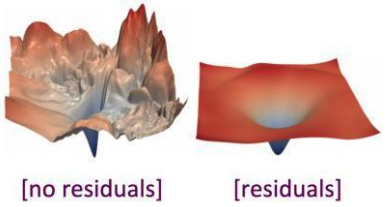
- Residual connections are a simple but powerful technique from computer vision.
- Therefore, directly passing "raw" embeddings to the next layer can actually be very helpful!

$$x_\ell = F(x_{\ell-1}) + x_{\ell-1}$$

- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.



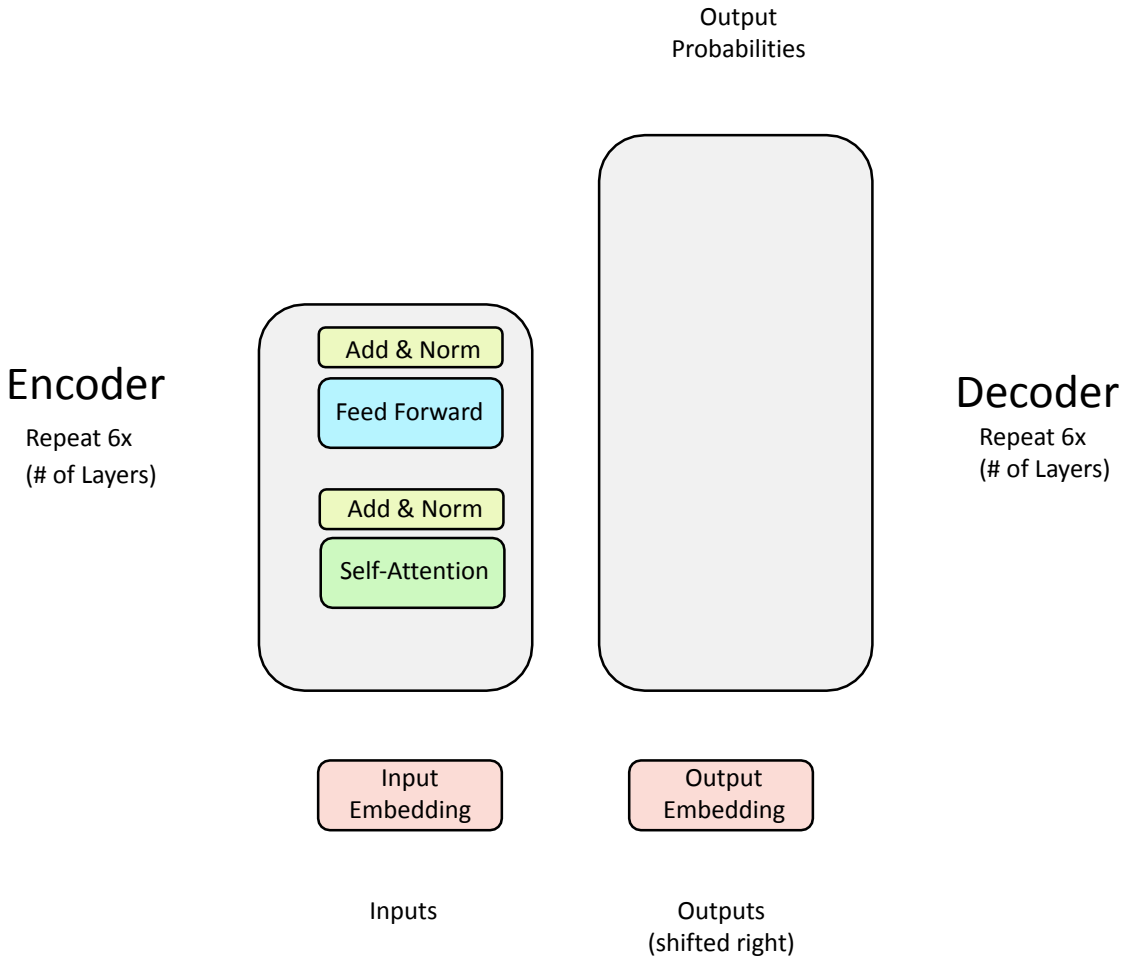
Residual connections are also thought to smooth the loss landscape and make training easier!



[Loss landscape visualization, Li et al., 2018, on a ResNet]

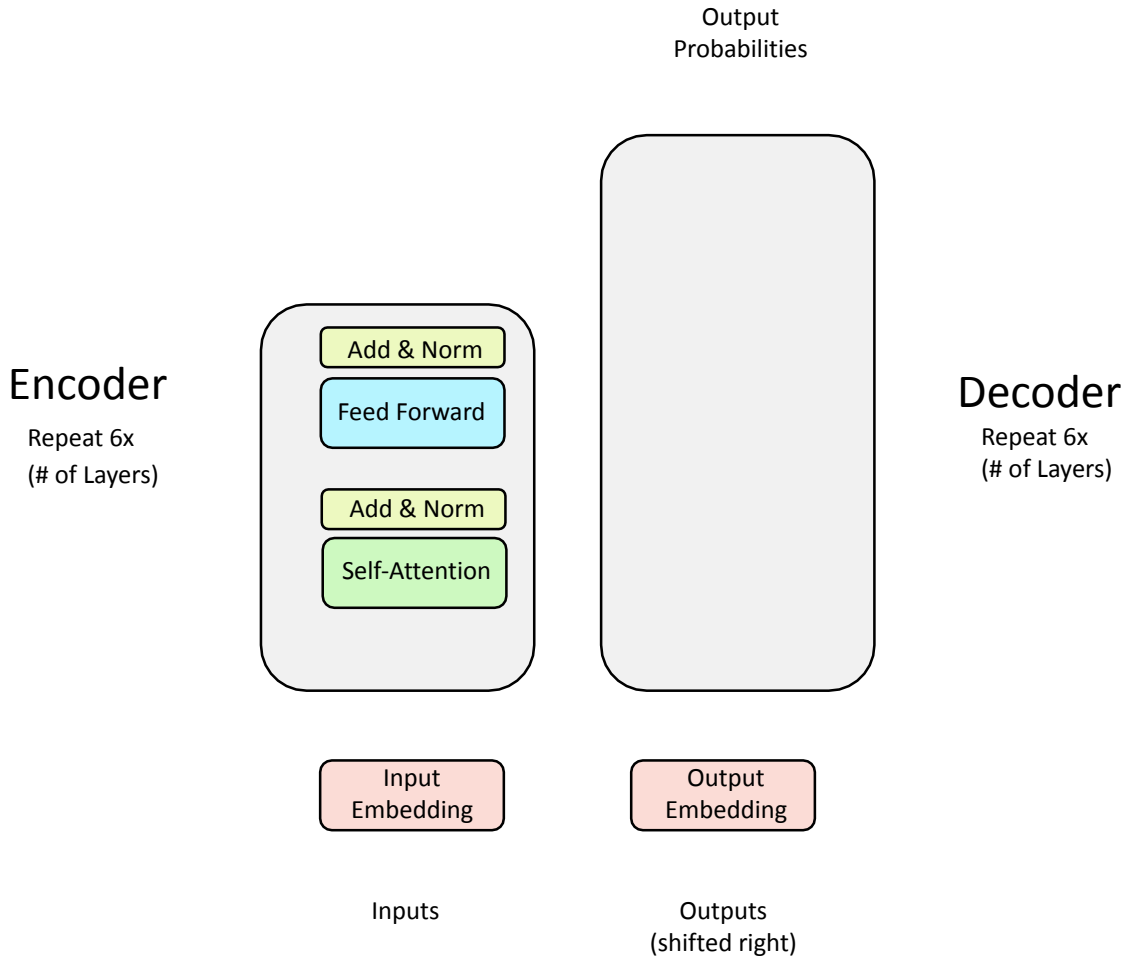
Training Trick #2: Batch Normalization

- **Problem:**
  - outputs from one or more of the intermediate layers can be abnormally large.
  - Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting.
- **Solution:** Reduce uninformative variation by **normalizing** to zero mean and standard deviation of one within each **Batch**.



Training Trick #3: Layer Normalization [Ba et al., 2016]

- **Problem:** Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting.
- **Solution:** Reduce uninformative variation by **normalizing** to zero mean and standard deviation of one within each **layer**.



Batch Normalization vs Layer Normalization

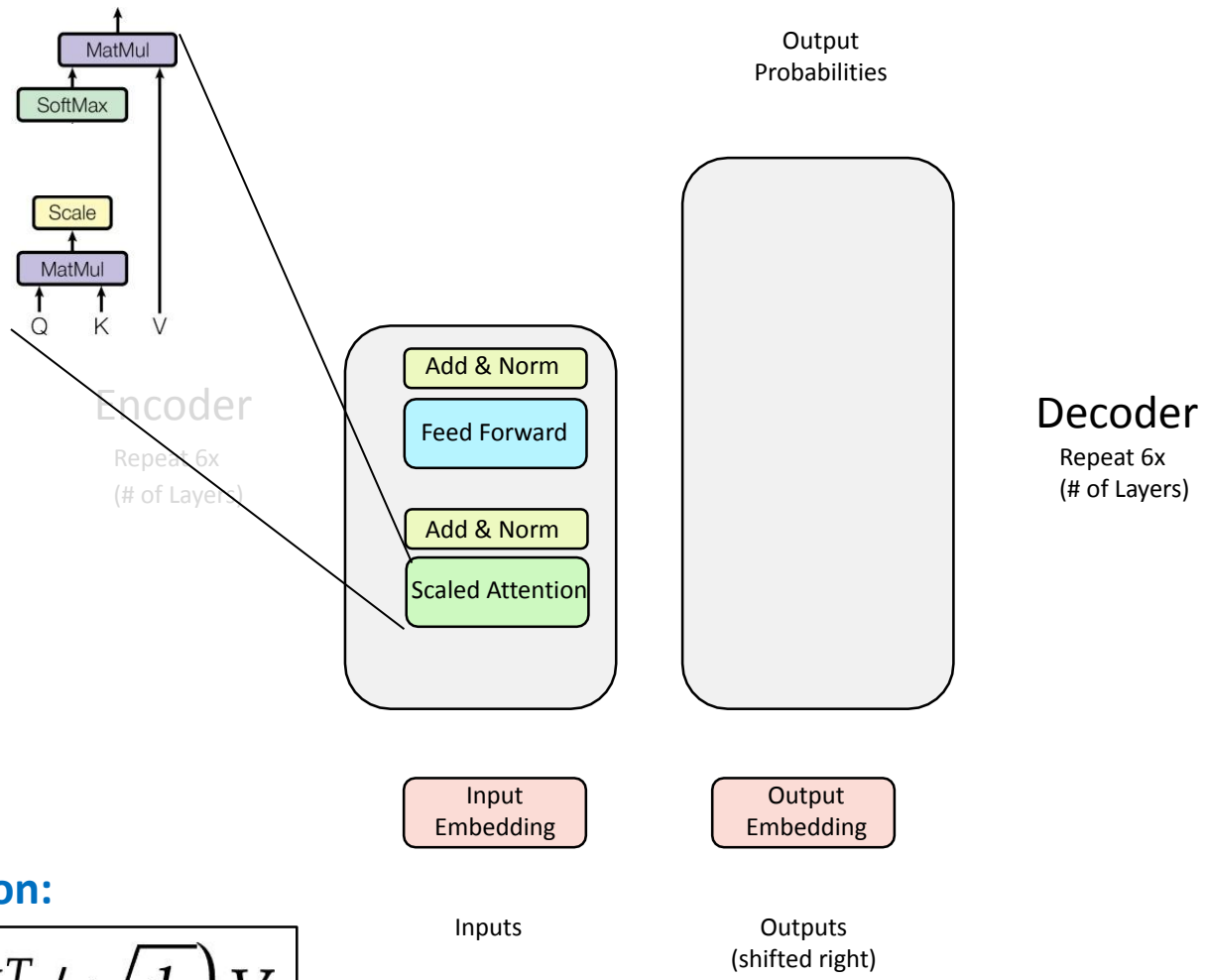
- Batch normalization normalizes each feature independently across the mini-batch. Layer normalization normalizes each of the inputs in the batch independently across all features.
- As batch normalization is dependent on batch size, it’s not effective for small batch sizes. Layer normalization is independent of the batch size, so it can be applied to batches with smaller sizes as well.
- Batch normalization requires different processing at training and inference times. As layer normalization is done along the length of input to a specific layer, the same set of operations can be used at both training and inference times.

Training Trick #4: Scaled Dot Product Attention

- After LayerNorm, the mean and variance of vector elements is 0 and 1, respectively.
- However, the dot product still tends to take on extreme values, as its variance scales with dimensionality d_k

Quick Statistics Review:

- Mean of sum = sum of means = $d_k * 0 = 0$
- Variance of sum = sum of variances = $d_k * 1 = d_k$
- To set the variance to 1, simply divide by $\sqrt{d_k}$



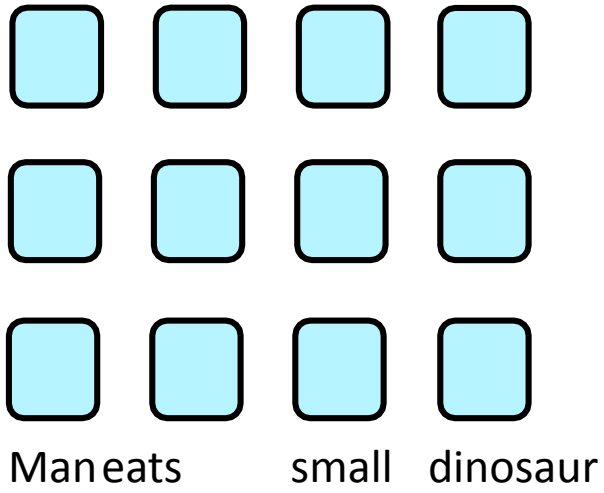
Updated Self-Attention Equation:

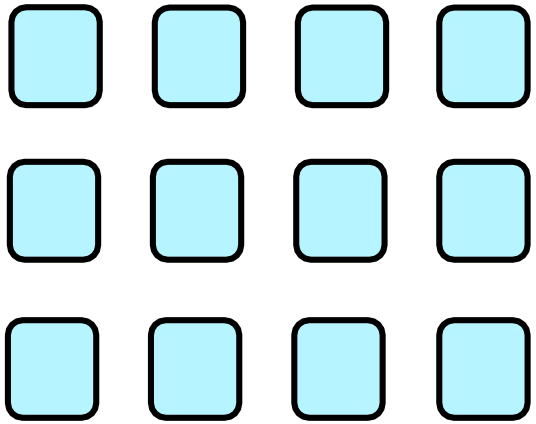
$$Output = softmax\left(QK^T / \sqrt{d_k}\right) V$$

Major issue!

- We're almost done with the Encoder, but we have a major problem!
- Consider this sentence:
  - "Man eats small dinosaur."
- Wait a minute, order doesn't impact the network at all!
- This seems wrong given that word order does have meaning in many languages, including English!

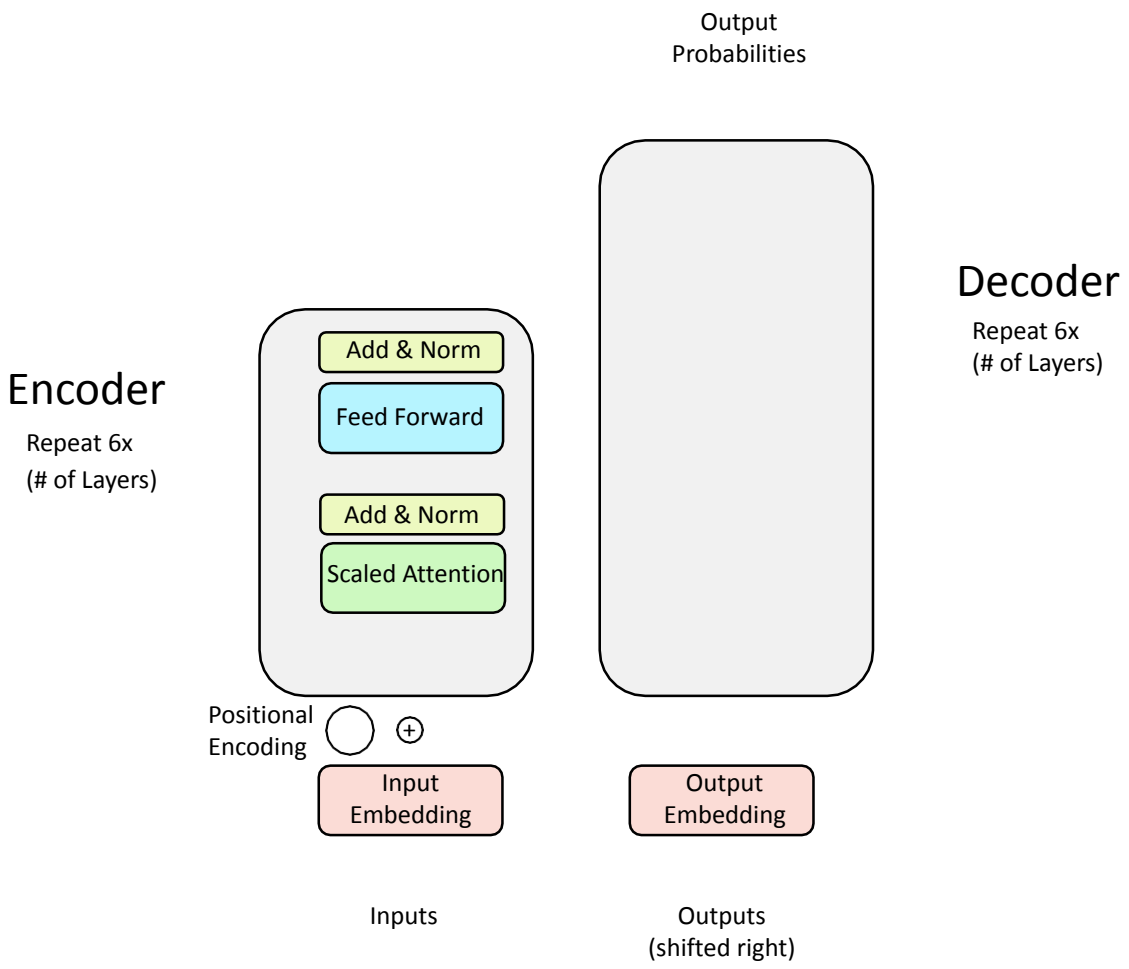
$$Output = softmax\left(QK^T / \sqrt{d_k}\right)V$$





Transformer-Based Encoder-Decoder Model

Solution: Inject Order Information through Positional Encodings!



Fixing the first self-attention problem: sequence order

- Since self-attention doesn't build in order information, we need to encode the order of the sentence in our keys, queries, and values.
- Consider representing each **sequence index** as a **vector**

$$p_i \in \mathbb{R}^d, \text{ for } i \in \{1,2, \dots, T\} \text{ are position vectors}$$

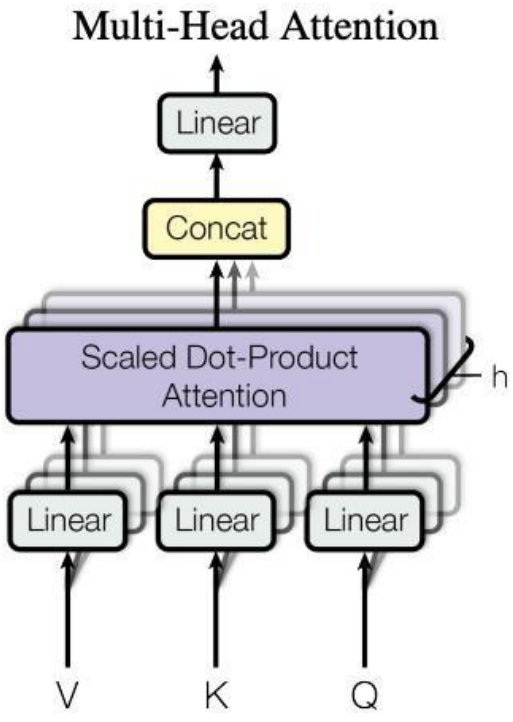
- Don't worry about what the p_i are made of yet!
- Easy to incorporate this info into our self-attention block: just add the p_i to our inputs!
- Let v_i, q_i, k_i old values, keys, and queries.

$$v_i = \tilde{v}_i + p_i$$
$$q_i = \tilde{q}_i + p_i$$
$$k_i = \tilde{k}_i + p_i$$

In deep self-attention networks, we do this at the first layer! You could concatenate them as well, but people mostly just add

Multi-Headed Self-Attention: k heads are better than 1!

- **High-Level Idea:** Let's perform self-attention multiple times in parallel and combine the results.



[Vaswani et al. 2017]

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

The Transformer Encoder: Multi-headed Self-Attention

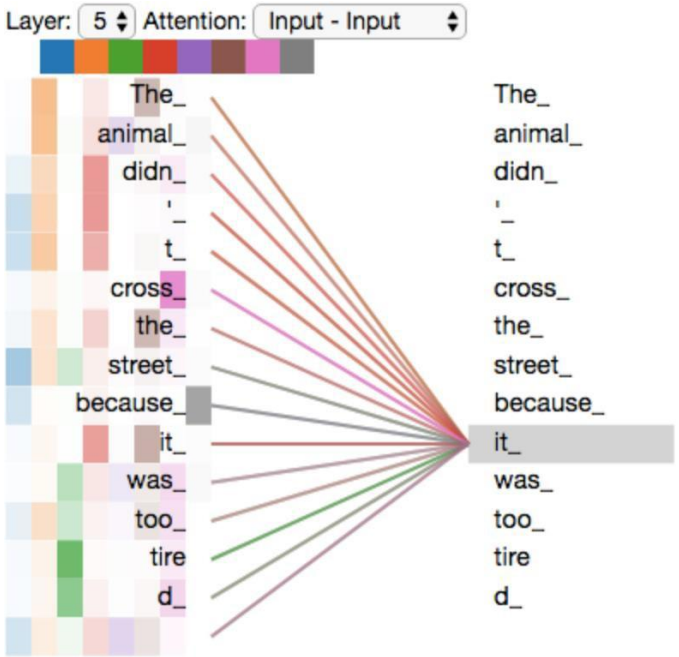
- What if we want to look in multiple places in the sentence at once?

i

- We'll define **multiple attention "heads"** through multiple Q,K,V matrices

d

- Let, $Q_\ell, K_\ell, V_\ell \in \mathbb{R}^{d \times \frac{d}{h}}$, where h is the number of attention heads, and ℓ ranges from 1 to h.
- Each attention head performs attention independently:
  - $\text{output}_\ell = \text{softmax}\left(\frac{XQ_\ell K^\top X^\top}{d_\ell}\right) * XV_\ell$, where $\text{output}_\ell \in \mathbb{R}^{d/h}$
- Then the outputs of all the heads are combined!
  - $\text{output} = Y[\text{output}_1; \ldots; \text{output}_h]$, where $Y \in \mathbb{R}^{d \times d}$



Credit to https://jalammar.github.io/illustrated-transformer/

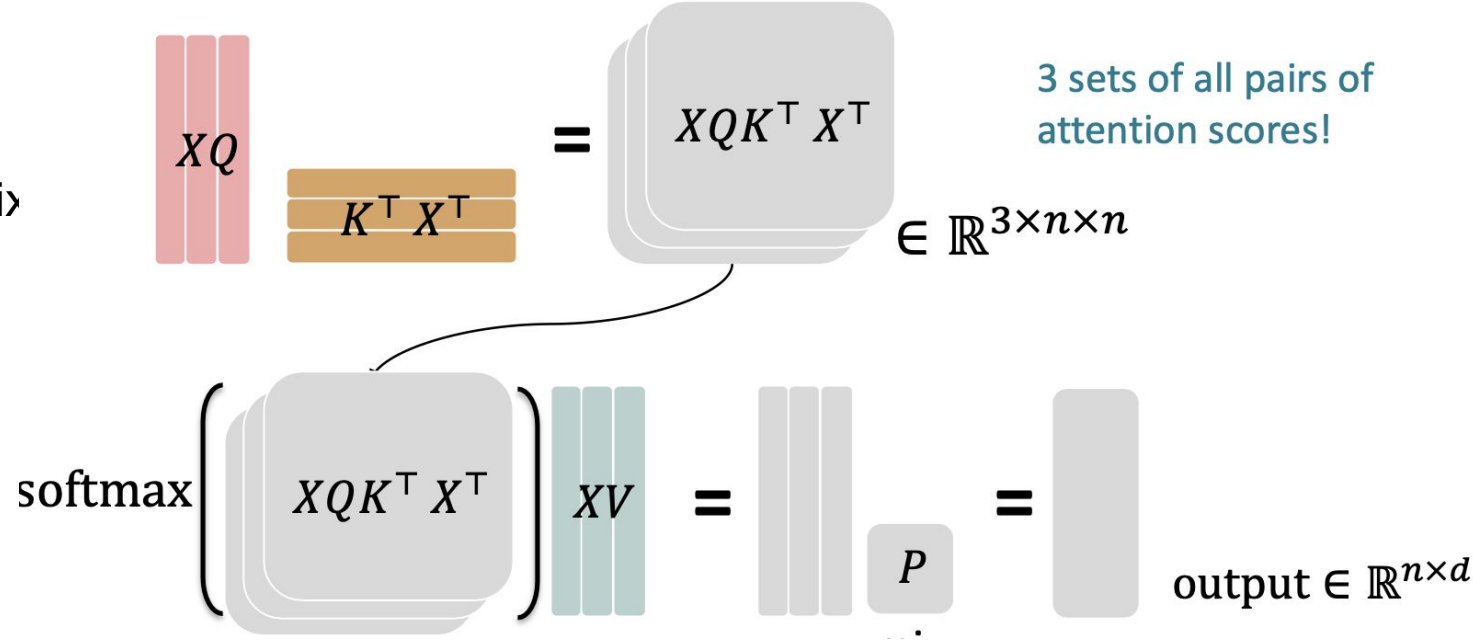
The Transformer Encoder: Multi-headed Self-Attention

Even though we compute h many attention heads, it's not really more costly.

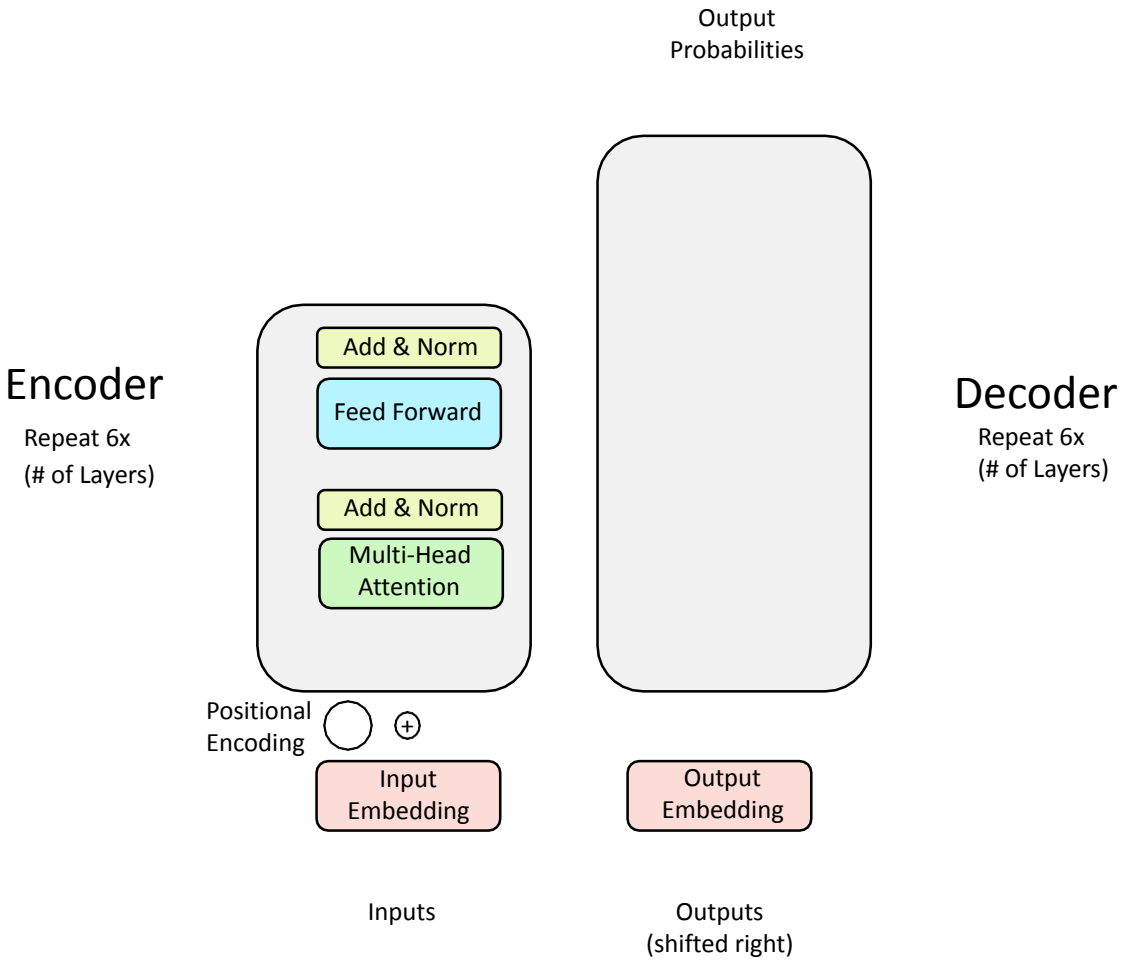
- We compute $XQ \in \mathbb{R}^{n \times d}$, and then reshape to $\mathbb{R}^{n \times h \times d/h}$. (Likewise for XK, XV.)
- Then we transpose to $\mathbb{R}^{h \times n \times d/h}$; now the head axis is like a batch axis.
- Almost everything else is identical, and the matrices are the same sizes.

First, take the query-key dot products in one matrix multiplication: $XQ\,XK^\top$

Next, softmax, and compute the weighted average with another matrix multiplication.

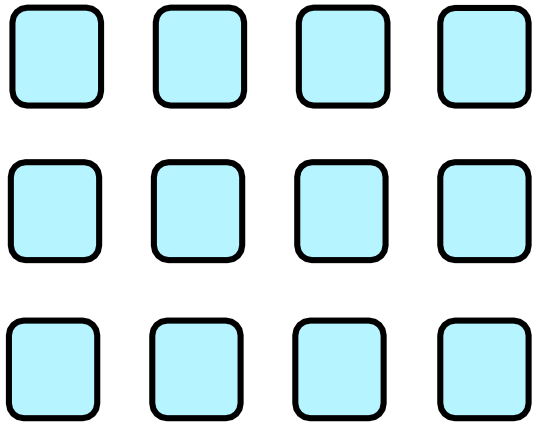


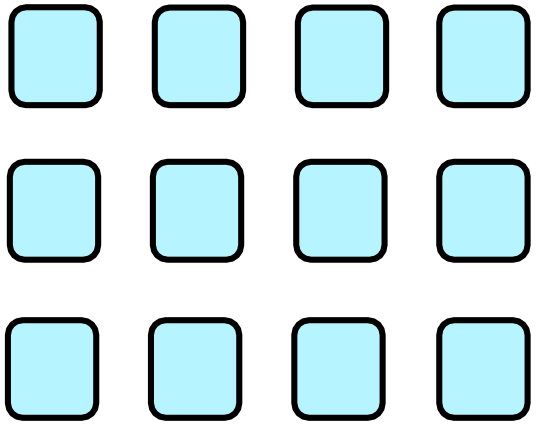
we've completed the Encoder! Time for the Decoder…



Decoder: Masked Multi-Head Self-Attention

- **Problem:** How do we keep the decoder from cheating? If we have a language modeling objective, can't the network just look ahead and "see" the answer?

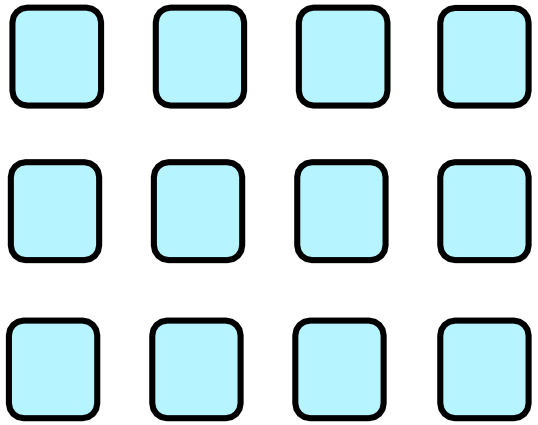


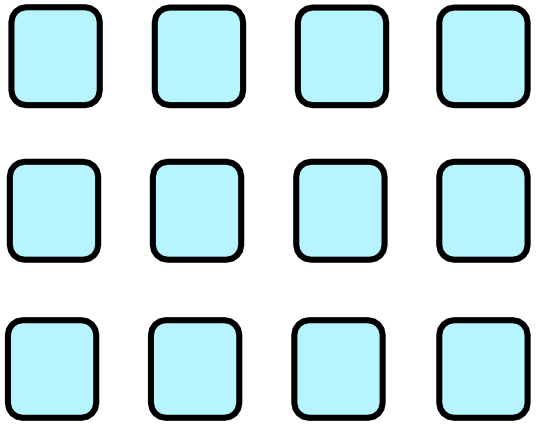


Transformer-Based
Encoder-Decoder Model

Decoder: Masked Multi-Head Self-Attention

- **Problem:** How do we keep the decoder from "cheating"? If we have a language modeling objective, can't the network just look ahead and "see" the answer?
- **Solution:** Masked Multi-Head Attention. At a high-level, we hide (mask) information about future tokens from the model.





Transformer-Based Encoder-Decoder Model

Masking the future in self-attention (Look Ahead Mask)

- To use self-attention in **decoders**, we need to ensure we can't peek at the future.
- At every timestep, we could change the set of **keys and queries** to include only past words. (Inefficient!)
- To enable parallelization, we **mask out attention** to future words by setting attention scores to $-\infty$.

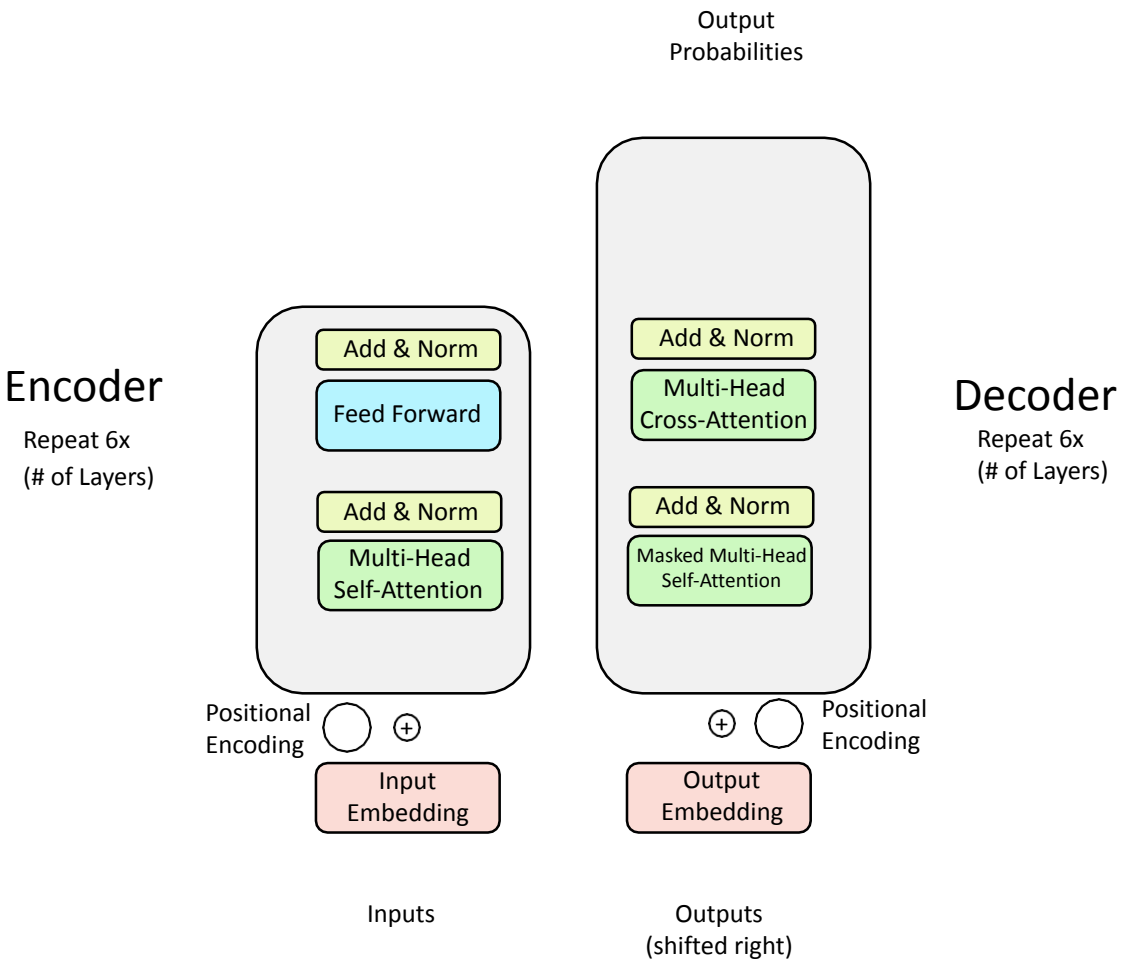
We can look at these (not greyed out) words

For encoding these words

	[START]	The	chef	who
[START]	$-\infty$	$-\infty$	$-\infty$	$-\infty$
The		$-\infty$	$-\infty$	$-\infty$
chef			$-\infty$	$-\infty$
who				$-\infty$

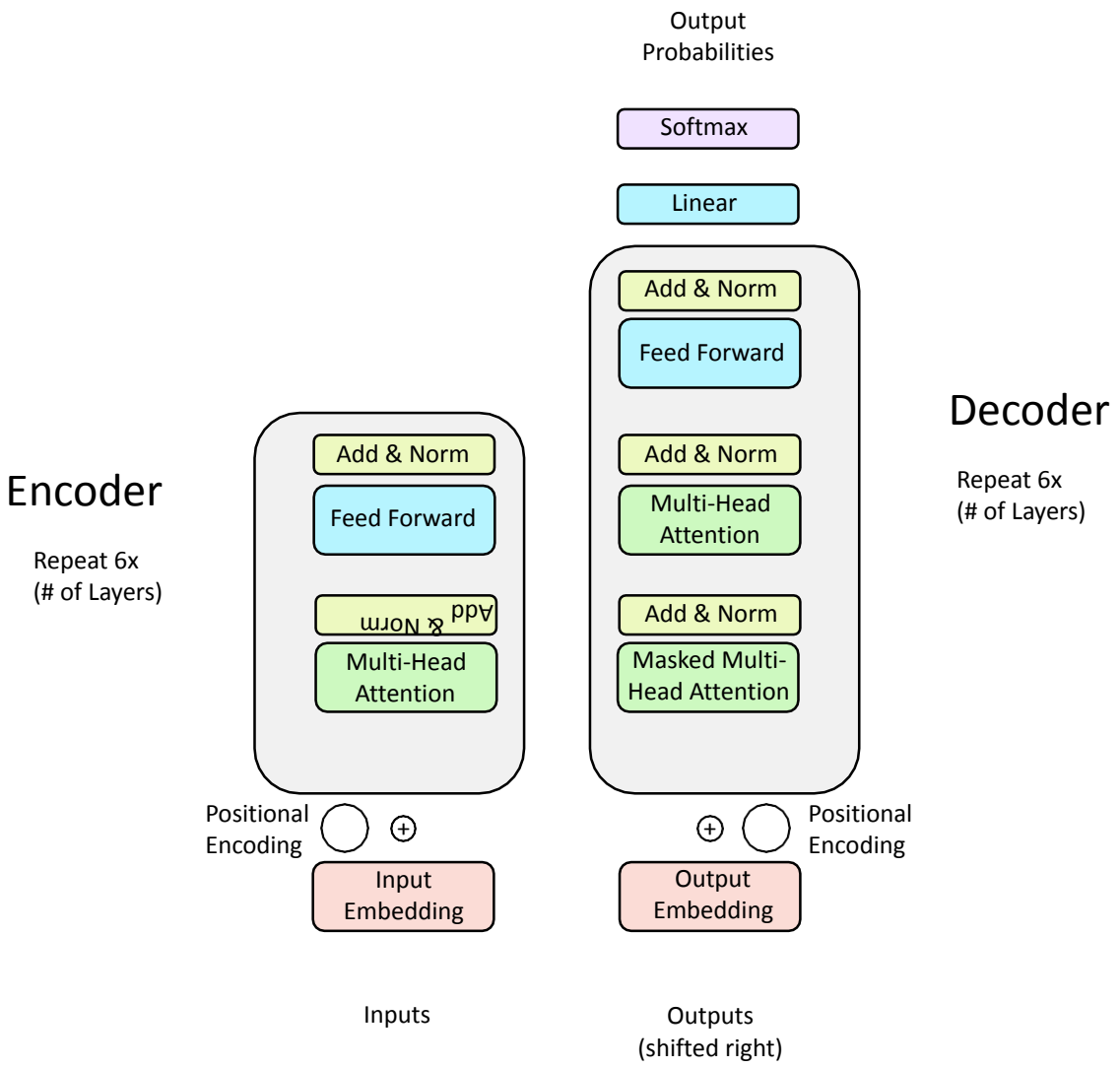
Encoder-Decoder Attention

- We saw that self-attention is when keys, queries, and values come from the same source.
- In the decoder, we have attention that looks more like what we saw last week.
- Let $h_1, \ldots, h_T$ be **output** vectors **from** the Transformer **encoder**; $x_i \in \mathbb{R}^d$
- Let $z_1, \ldots, z_T$ be input vectors from the Transformer **decoder**, $z_i \in \mathbb{R}^d$
- Then keys and values are drawn from the **encoder** (like a memory):
  - $k_i = Kh_i, v_i = Vh_i$.
- And the queries are drawn from the **decoder**, $q_i = Qz_i$.



Decoder: Finishing touches!

- Add a feed forward layer (with residual connections and layer norm)
- Add a final linear layer to project the embeddings into a much longer vector of length vocab size (logits)
- Add a final softmax to generate a probability distribution of possible next words!



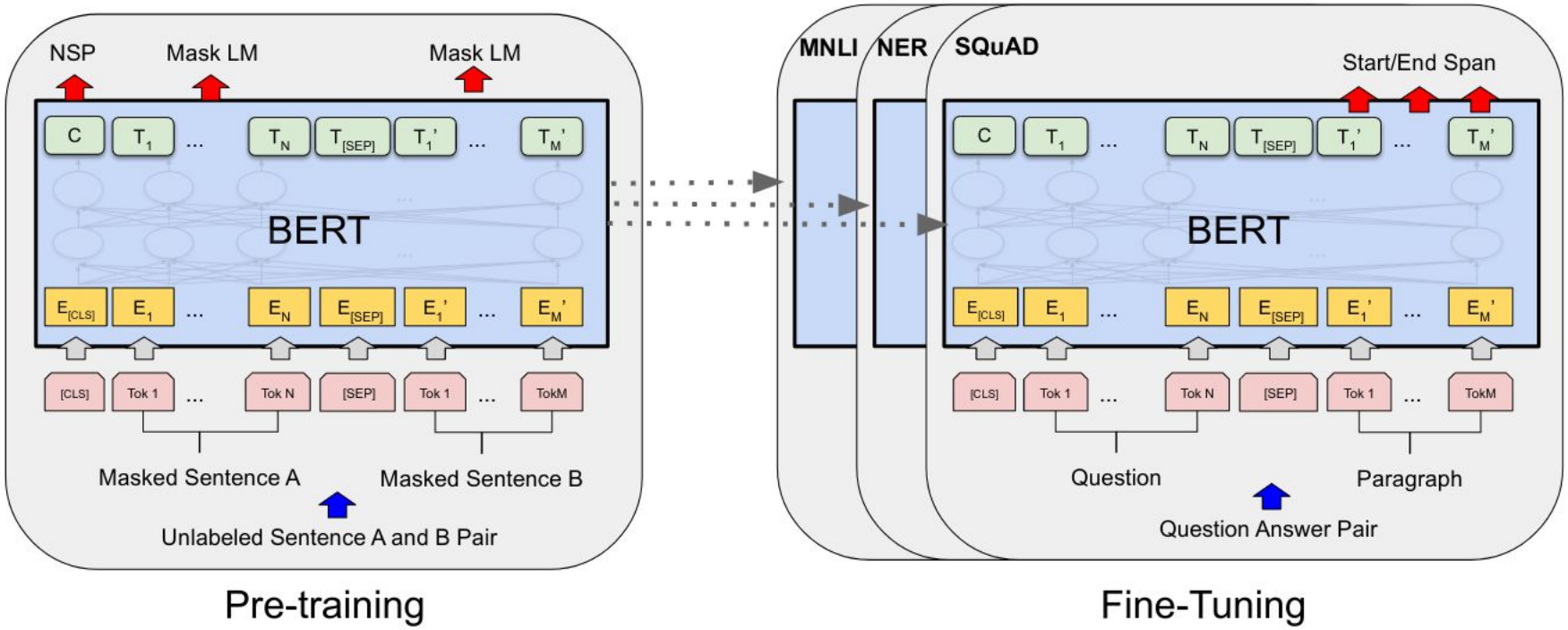
Training Tips

- Byte-pair encodings
- ADAM optimizer with learning rate changes
- Dropout during training at every layer just before adding residual
- Auto-regressive decoding
  - Use previous time step output as input
- Beam search and length penalties (beam size = 4, α=0.65)
  - Short utterances are favored in MT, so scores (log-probabilities) are divided by $length^{\alpha}$

BERT (Bidirectional Encoder Representations from Transformers)

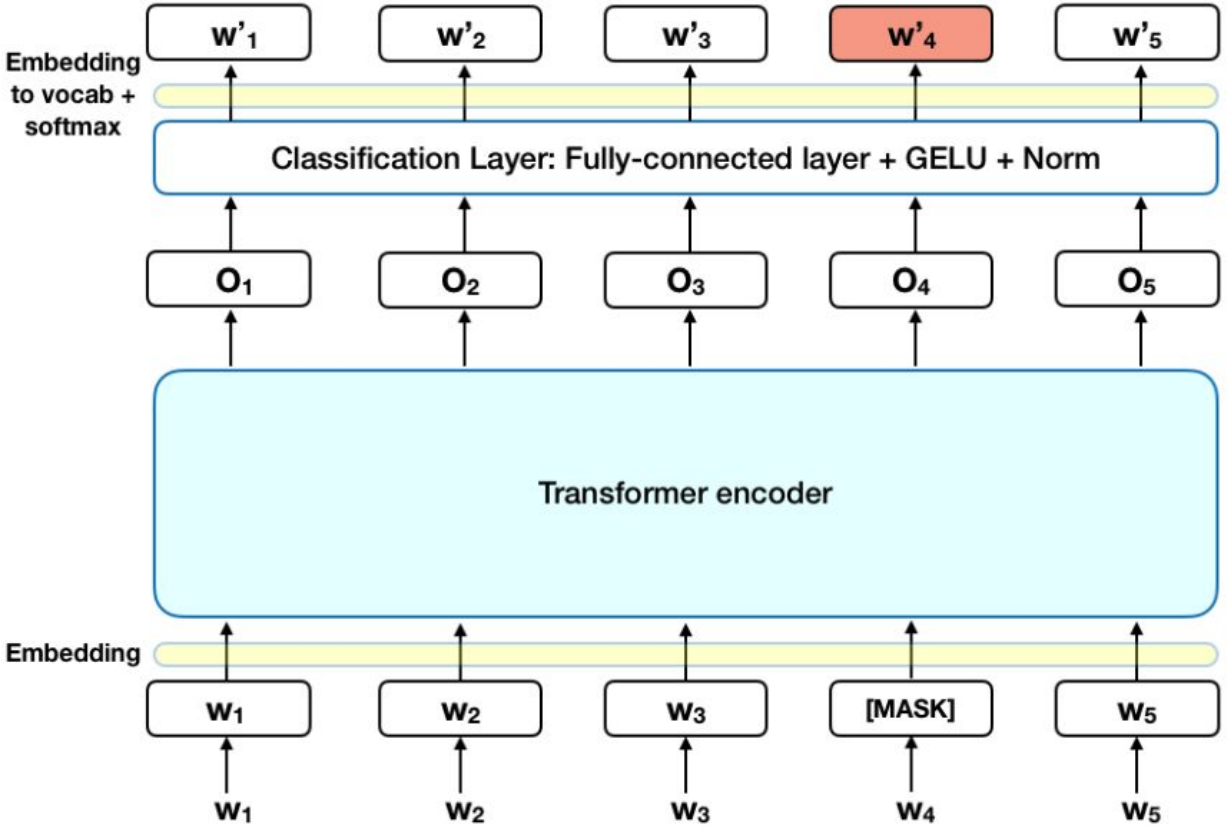
- Bidirectional training of Transformer
- The paper's results show that a language model which is bidirectionally trained can have a deeper sense of language context and flow than single-direction language models.
- In the paper, the researchers detail a novel technique named Masked LM (MLM)
- Since BERT's goal is to generate a language model, only the encoder mechanism.

Overview



BERT leverages a fine-tuning based approach for applying pre-trained language models
A common architecture is trained for a relatively generic task.
It is fine-tuned on specific downstream tasks that are more or less similar to the pre-training task.

Masked LM (MLM)



- 80% of the time: Replace the word with the `[MASK]` token, e.g., `my dog is hairy` → `my dog is [MASK]`
- 10% of the time: Replace the word with a random word, e.g., `my dog is hairy` → `my dog is apple`
- 10% of the time: Keep the word unchanged, e.g., `my dog is hairy` → `my dog is hairy`. The purpose of this is to bias the representation towards the actual observed word.

Masked LM Example from the Paper

Byte Pair Encoding (BPE) and WordPiece Subword tokenization

Origin:

BPE: Originally developed for data compression, adapted for NLP by Rico Sennrich et al.
WordPiece: Developed by Google, initially for speech recognition.

Merging criterion:

BPE: Merges based on frequency of adjacent pairs.
WordPiece: Merges based on maximizing the likelihood of the training data.

Handling of word beginnings:

BPE: Typically treats word beginnings the same as other subwords.
WordPiece: Often uses a special token (e.g., "##") to mark subwords that don't start words
.

Vocabulary building:

- BPE: Builds vocabulary bottom-up, starting from characters.
- WordPiece: Also bottom-up, but with different selection criteria.

Used in:

- BPE: GPT, RoBERTa
- WordPiece: BERT, DistilBERT

Handling rare words:

Both are effective at breaking down rare or unknown words into subwords.

BPE Example

A text corpus with the following four words: “ab”, “bc”, “bcd”, and “cde”.
The initial vocabulary consists of all the bytes or characters in the text corpus: {“a”, “b”, “c”, “d”, “e”}.

Step 1: Initialize the vocabulary
Vocabulary = {"a", "b", "c", "d", "e"}

Step 2: Calculate the frequency
Frequency = {"a": 1, "b": 3, "c": 3, "d": 2, "e": 1}

Step 3a: Find the most frequent pair of two characters
The most frequent pair is "bc" with a frequency of 2.
Step 3b: Merge the pair
Merge "bc" to create a new subword unit "bc".
Step 3c: Update frequency counts
Update the frequency counts of all the bytes or characters that contain “bc”:
Frequency = {"a": 1, "b": 2, "c": 3, "d": 2, "e": 1, "bc": 2}
Step 3d: Add the new subword unit to the vocabulary
Add “bc” to the vocabulary:
Vocabulary = {"a", "b", "c", "d", "e", "bc"}
Repeat steps 3a-3d until the desired vocabulary size is reached.

Step 4: Represent the text corpus using subword units
The resulting vocabulary consists of the following subword units: {"a", "b", "c", "d", "e", "bc", "cd", "de","ab","bcd","cde"}

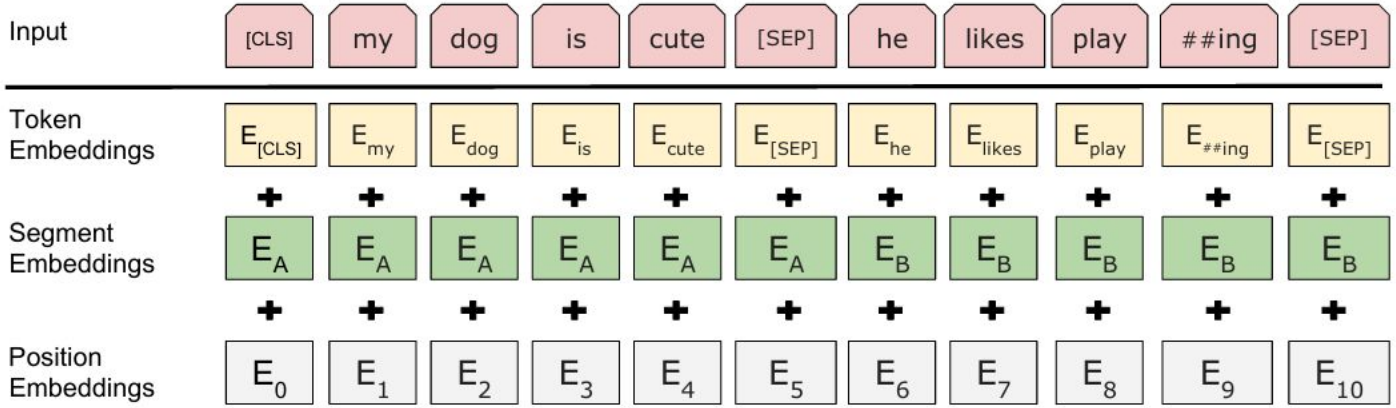
Next Sentence Prediction (NSP)

- There are two input sequences (separated using [SEP] token, and **Segment Embeddings** are used).
- It is a binary classification task involving prediction to tell if the second sentence succeeds the first sentence in the corpus.

Input = [CLS] the man went to [MASK] store [SEP]
he bought a gallon [MASK] milk [SEP]
Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]
penguin [MASK] are flight ##less birds [SEP]
Label = NotNext

Next Sentence Prediction (NSP)



To predict if the second sentence is indeed connected to the first, the following steps are performed:

1. The entire input sequence goes through the Transformer model.
2. The output of the [CLS] token is transformed into a 2×1 shaped vector, using a simple classification layer (learned matrices of weights and biases).
1. Calculating the probability of IsNextSequence with softmax.

When training the BERT model, Masked LM and Next Sentence Prediction are trained together, with the goal of minimizing the combined loss function of the two strategies.

How to use BERT (Fine-tuning)

1. Classification tasks such as sentiment analysis are done similarly to Next Sentence classification, by adding a classification layer on top of the Transformer output for the [CLS] token.
2. In Question Answering tasks (e.g. SQuAD v1.1), the software receives a question regarding a text sequence and is required to mark the answer in the sequence.

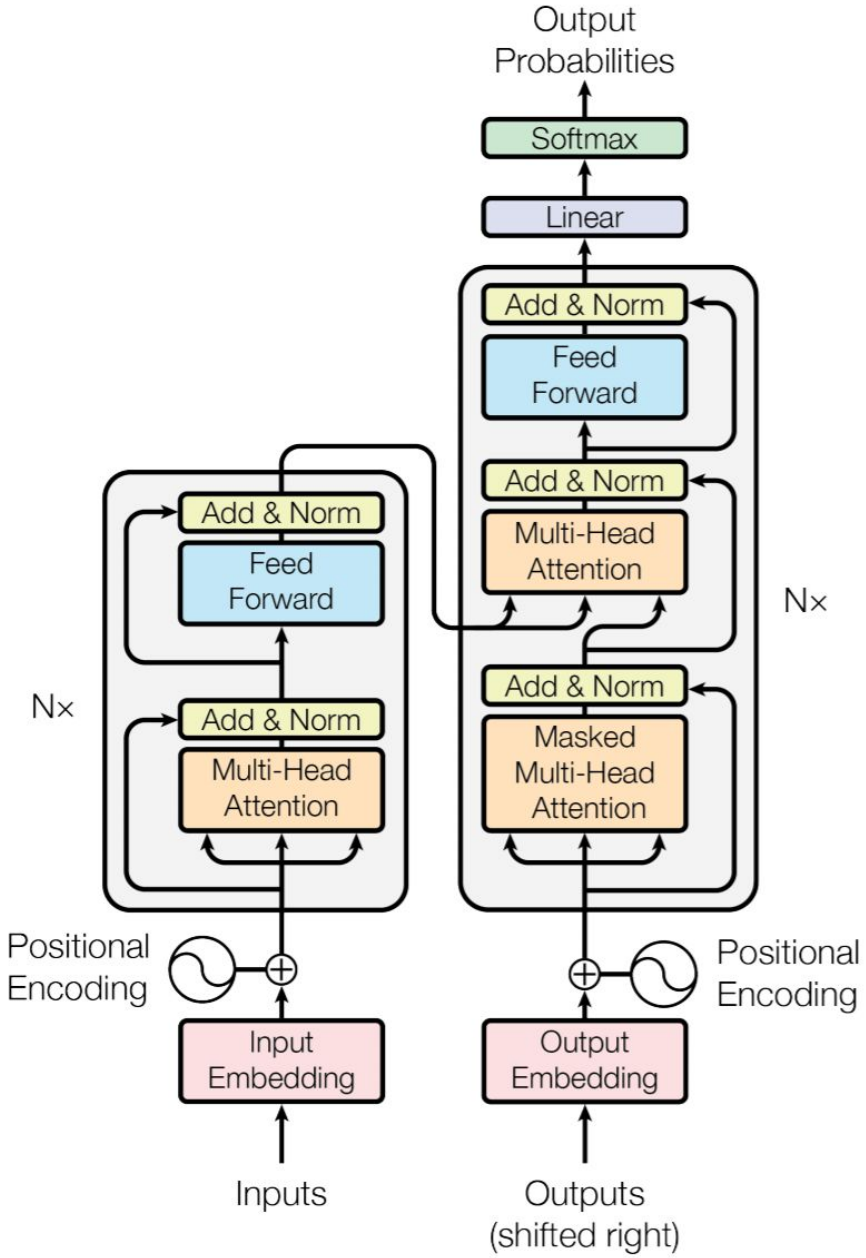
   Using BERT, a Q&A model can be trained by learning two extra vectors that mark the beginning and the end of the answer.
3. In Named Entity Recognition (NER), the software receives a text sequence and is required to mark the various types of entities (Person, Organization, Date, etc) that appear in the text.

Takeaway

1. *Model size matters, even at huge scale.* BERT_large, with 345 million parameters, is the largest model of its kind. It is demonstrably superior on small-scale tasks to BERT_base, which uses the same architecture with "only" 110 million parameters.
2. *With enough training data, more training steps == higher accuracy.* For instance, on the MNLI task, the BERT_base accuracy improves by 1.0% when trained on 1M steps (128,000 words batch size) compared to 500K steps with the same batch size.
3. *BERT's bidirectional approach (MLM) converges slower than left-to-right approaches* (because only 15% of words are predicted in each batch) but bidirectional training still outperforms left-to-right training after a small number of pre-training steps.

Concluding Remarks

- **Non-recurrence** model is easy to parallelize
- **Multi-head attention** captures different aspects by interacting between words
- **Positional encoding** captures location information
- Each transformer block can be applied to diverse tasks



Continuing Contextual Language Models

- - Contextual language models process input in context to capture nuanced meanings.
- - Key advancements include leveraging attention mechanisms and bidirectional representations.
- - Designed to handle diverse tasks such as translation, summarization, and Q&A.

GPT and GPT-2

- GPT (Generative Pre-trained Transformer):
- - Autoregressive model trained on massive datasets.
- - Focused on generating coherent and contextually relevant text.
- GPT-2:
- - An enhanced version with more parameters.
- - Generates human-like text and completes sentences effectively.

BART

- - Bidirectional and Autoregressive Transformer.
- - Designed for denoising tasks by reconstructing corrupted input.
- - Excels in summarization, translation, and generative tasks.

ELECTRA

- - Efficient pretraining method using replaced token detection.
- - Trains a discriminator to detect replaced tokens rather than generating text.
- - Results in faster training with competitive performance.

Features that Differentiate Models

- - Training Objectives:
- * GPT: Autoregressive; Predicts next token.
- * BERT: Bidirectional; Masked token prediction.
- * ELECTRA: Replaced token detection.
- - Model Architectures:
- * Differences in layers, attention mechanisms, and encoding strategies.

2024

1. Large Language Models (LLMs) and their applications:
    1. Scaling up models to trillions of parameters
    2. Improving efficiency and reducing computational costs
    3. Exploring few-shot and zero-shot learning capabilities
2. Multimodal AI:
    1. Integrating vision and language models
    2. Developing models that can understand and generate text, images, and video
    3. Exploring cross-modal transfer learning
3. Responsible AI and Ethical Considerations:
    1. Addressing bias and fairness in NLP models
    2. **Developing more interpretable and explainable models**
    3. Ensuring privacy and security in language technologies
    4. Investigating sparse and **mixture-of-experts models**
4. Multilingual and Cross-lingual NLP:
    1. Creating models that can understand and generate multiple languages
    2. Improving low-resource language processing
    3. Developing universal language models
5. Conversational AI and Dialogue Systems:
    1. Enhancing open-domain dialogue capabilities
    2. Improving task-oriented dialogue systems
    3. Developing more natural and context-aware conversational agents
6. Reasoning and Cognitive Abilities in NLP:
    1. Enhancing logical reasoning capabilities in language models
    2. Improving common sense reasoning and world knowledge integration
    3. **Exploring analogical reasoning and abstract thinking in AI**